Gain scheduling and Linear interpolation based on velocity for an adaptive trajectory control of a self-driving bike

SSY226 — Design project in systems, control and mechatronics

Ayman Alzein & Armin Khorsandi

***Abstract*—This paper presents a project focused on improving the control system of a self-driving bicycle used in automotive safety testing. The bicycle is designed to autonomously maintain balance and follow a predefined trajectory. The primary objective was to implement gain scheduling and system adaptation based on the bicycle's velocity, incorporating linear interpolation to mitigate abrupt changes in control gains.**

Key components in this project include trajectory controller using a Linear Quadratic Regulator (LQR) and state estimation using a Kalman filter. Gain scheduling is applied to both the Kalman filter and the feedforward contribution from the heading change reference to the trajectory controller, while the LQR remains invariant to changes in velocity. Linear interpolation further refines the control system by smoothing transitions between different gain values, reducing abrupt changes.

Simulation results demonstrate that velocity-adaptive control significantly improves the bicycle's performance in following the pre-defined path, reducing lateral and heading errors. These findings suggest that the integration of gain scheduling and linear interpolation substantially enhances path-tracking accuracy across varying speeds, contributing to the overall effectiveness of the control system.

I. Introduction

A. Background:

Automotive companies are in a race to develop reliable autonomous vehicles, which require rigorous testing. One critical test scenario involves a physical bike with a dummy crossing the path of a self-driving car to analyze the car's response. The Autobike Project is a collaborative effort between Chalmers University, Mälardalen University, and several industry partners, aimed at developing autonomous bicycles capable of following predefined paths with high precision and consistency. These autonomous bicycles are designed for use in test-driving scenarios, where they carry a dummy to support the testing and evaluation of automotive safety systems. The bike is capable of self-balancing and following a predetermined path at a constant, predefined reference velocity. The path is characterized by positions in a 2D Cartesian coordinate system, with corresponding reference heading angles for each point. This serves as the basis for the work presented in this paper.

B. List of symbols and units

- ψ: Heading angle of the bike in the world frame $[rad]$
- ψ_{ref}: Desired heading angle of the bike in the world frame $[rad]$
- $\dot{\psi}$: Heading angle rate $[rad/s]$
- φ: Roll angle [rad]
- φ_{ref}: Desired roll angle for balancing the bike [rad]
- $\dot{\varphi}$: Roll angle rate $[rad/s]$
- δ: Steering angle [rad]
- $\dot{\delta}$: Steering angle rate $[rad/s]$
- v: Bike's current velocity $[m/s]$
- v: Desired bike's velocity $[m/s]$
- X: Bike position in Y on the world frame $[m]$
- X_{ref}: Desired bike position in X on the world frame $[m]$
- Y: Bike position in Y on the world frame $[m]$
- Y_{ref}: Desired bike position in Y on the world frame $[m]$
- e_1: Lateral error $[m]$
- e_2: Heading error $[rad]$
- ω_{motor}: Forward motor's rotational speed $[rad/s]$
- G_{bike}: Bike model transfer function
- Q: Weight matrix for states
- R: Weight matrix for inputs
- S: Riccati equation solution matrix
- lr: Distance from rear wheel of the bike to frame's center of mass $[m]$
- lf: Distance from front wheel of the bike to frame's center of mass $[m]$
- Δ_{ref}: ILC correction on reference signals

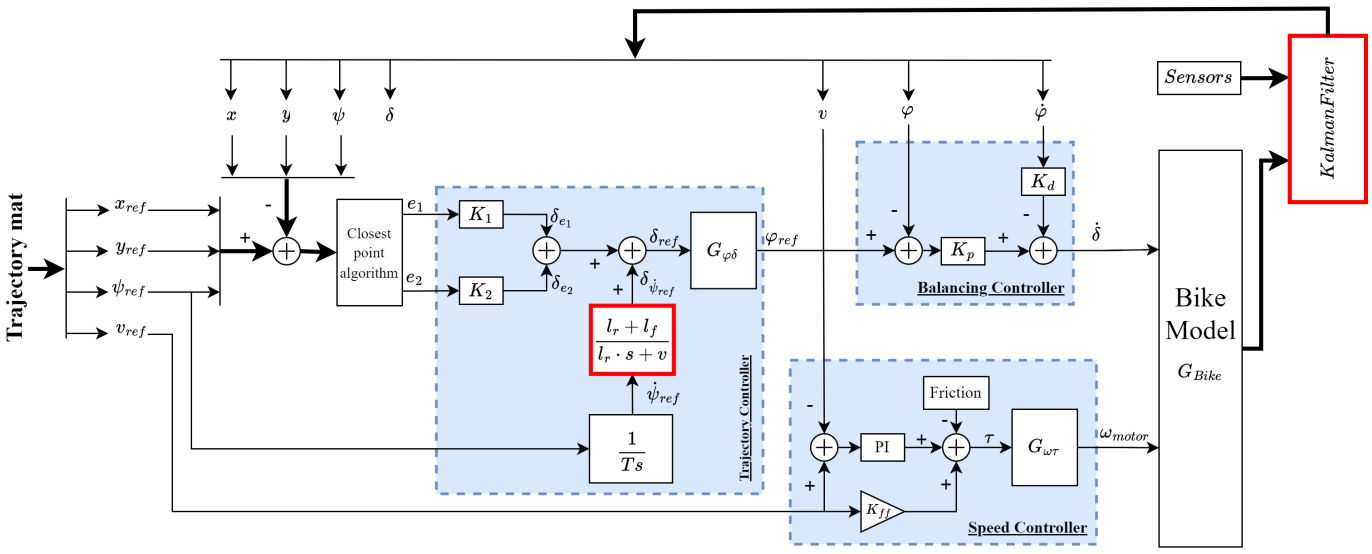


Fig. 1. Bike Simulink model. Red outlines indicate dependence on velocity.

C. System overview

The bike is equipped with two motors: one connected to the steering shaft, which controls the steering angle, and another responsible for driving the wheel. It also features a control box housing a myRIO controller, where all the algorithms are stored and executed. Additionally, the bike is equipped with an IMU, GPS, and a router, enabling real-time wireless data transfer to monitor the bike's parameters during operation.

The entire system is built on a generalized bike model, allowing the control box and motors to balance any bike, provided the parameters are correctly tuned and inputted into the main code.

To operate the bike, a set of files must be uploaded to the myRIO. The main code requires access to two specific files on myRIO:

- The parameter file (matrix-mat), which contains the control parameters specifically generated for a particular bike. These parameters include the bike's dimensions and controller gains.
- The trajectory file, which consists of a large matrix where each row contains 2D GPS coordinates, the heading angle, and velocity references.

At each time step, the bike evaluates the closest reference coordinates from the trajectory file relative to its current position to determine the direction and speed it should follow. This evaluation uses the closest point algorithm, which then sends the extracted reference values to the control loop. These references are illustrated on the far left of the block diagram in Figure 1.

Additionally, the same algorithm calculates the lateral distance error from the reference GPS coordinates, e_1, and the heading angle error, e_2. These two errors are fed into the trajectory controller, where an LQR controller computes the corresponding steering angle reference, δ_{ref}.

In the trajectory controller, an additional contribution from the steering rate is also added to the steering angle reference, δ_{ref}. This contribution utilizes a transfer function, shown in the red block within the trajectory controller in Figure 1, which depends on the current velocity of the bike.

Since the bike operates at varying speeds, this transfer function must adapt to the current velocity. This transfer function is typically generated in Matlab, saved as a state space in the matrix-mat file, and then uploaded to the myRIO. To make the state space adaptive, there are two options:

- Generate multiple state spaces corresponding to different velocities in Matlab and schedule them in LabVIEW based on the current speed.
- Implement the state space calculation directly in LabVIEW. '

To maintain consistency with the standards in the Autobike project, the first option was more feasible and was therefore implemented.

Moving on, the steering angle reference, δ_{ref}, is converted to a roll reference, φ_{ref}, and is sent from the trajectory controller to the balancing loop, as shown in Figure 1. The balancing controller is a PD controller that outputs a steering angular velocity or steering rate, $\dot{\delta}$.

The speed controller operates separately from the trajectory and balancing loops. It is a PI controller with an open-loop feedforward term.

Further to the right in Figure 1, the bike model, G_{Bike}, is used in the simulation software to evaluate different control strategies before they are implemented on the actual bike.

The sensor block, located far up to the right in Figure 1, mimics the actual sensors characteristics by adding noise and including bias to the true values before sending them to the next block, the Kalman Filter.

The Kalman Filter estimates the bike's states based on the sensors values and operates in two stages: prediction of

the next states and updating the estimation. These stages require pre-determined gains and system matrices, which are generated in Matlab and included in the matrix-mat file sent to the actual bike. Since these matrices and gains depend on the current velocity, they are aslo scheduled in LabVIEW.

Further details about each part of the system are described in the following sections.

1) Closest point algorithm: The reference trajectory matrix is a large dataset that includes the x and y coordinates, reference heading angles, and velocities at each point along the path. For the controller to function properly, the bike needs to identify the closest reference point and determine the next point it should move toward, along with the corresponding velocity.

To find the closest reference point, the algorithm calculates the distance between the bike's current position and each point in the trajectory. Once the closest point is identified, the bike uses the next point in the sequence as its reference. To prevent the bike from making erratic turns, especially when the reference points form a cross, the algorithm always creates a smaller local subset of the trajectory matrix. It checks for the closest point and the next reference point within this subset before shifting it forward along the original trajectory matrix.

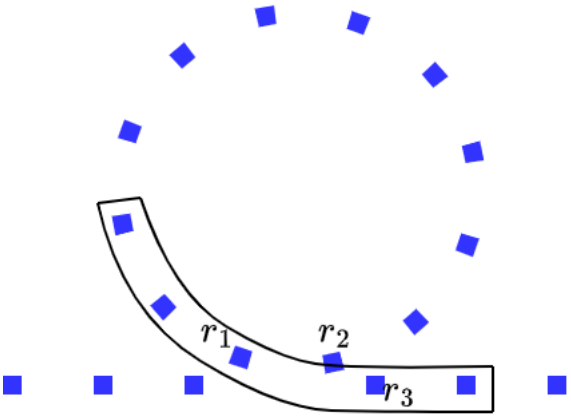


Fig. 2. Assuming the blue points belong to the trajectory matrix, when the bike completes the circle and arrives at r_1, it will continue straight to r_3 instead of doing another lap towards r_2, thanks to the local reference subset.

The outputs of the closest point algorithm are:

- Lateral error, e_1, which represents the distance between the current bike position and the closest point on the reference trajectory.
- Heading error, e_2, which is the difference between the reference heading angle, ψ_{ref}, at the closest point and the current heading angle.

2) Trajectory control: At each time step, the inputs to the trajectory controller are the lateral and heading errors, e_1 and e_2, as illustrated as signals in the block diagram in Figure 1, and for clarification shown here in figure 3.

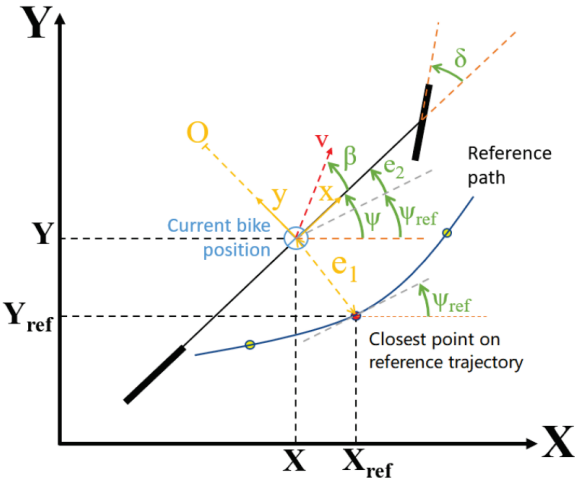


Fig. 3. A top view of the balanced bike is shown, where its current position is at (X, Y), and the reference point, according to the closest point algorithm, is at (X_{ref}, Y_{ref}). The lateral error, e_1, represents the deviation between the bike's position and the reference point, while the heading error, e_2, indicates the angular difference between the bike's current heading and the reference heading at (X_{ref}, Y_{ref}).[1]

The controller then outputs the reference steering angle, which is determined by considering several contributing factors. To minimize the lateral and heading errors, an LQR controller is employed, as described in [1]. Additionally, a feedforward term is introduced to enhance the bike's responsiveness to heading changes from the predefined path. This feedforward contribution ensures that the bike does not solely react to increases in lateral and heading errors but also proactively adjusts based on the expected heading change from the reference path. This anticipatory adjustment enables faster responses to heading changes, thereby improving trajectory control.

The reference steering angle is thus composed of the sum of these three contributions at each time step, as shown below.

$$\delta = \delta_{e_1} + \delta_{e_2} + \delta_{\dot{\psi}_{ref}} \tag{1}$$

Finally, since the input to the balancing controller is a roll reference, the reference steering angle is converted into a roll reference using a transfer function $G_{\varphi\delta}$.

3) Balancing controller: The balancing controller is responsible for maintaining the bike's balance. It takes the roll reference, φ_{ref}, as an input and controls the steering rate, $\dot{\delta}$. Essentially, it operates as a PD controller, under the assumption that the roll rate reference is zero, $\dot{\varphi}_{ref} = 0$.

4) Speed controller: Another component of the system is the speed controller. The speed controller receives the velocity reference from the trajectory matrix using the closest point algorithm at each time step, along with the current velocity, and uses a PI controller to regulate the speed. A feedforward term is also introduced to compensate for ground friction, aiding the bike in initiating movement from a standstill. The output of the speed controller is the forward motor's rotational

speed, ω_{motor}, which is then converted to the bike's linear velocity through the bike model G_{bike}.

5) State Estimation and Kalman Filter: The states outputted from the bike model (G_{bike}), along with the sensor signals, are sent to a Kalman filter. By performing prediction and update steps, the Kalman filter estimates the states and measurements, which are then sent to the appropriate components of the system to be used in the next time step.

II. Methodology

The algorithms that are alredy implemented in MATLAB/Simulink and LabVIEW/C allows the bikes to self-balance and follow various paths while maintaining a constant velocity. This is because the bike's dynamics are optimized for a specific constant speed, with gains, such as Kalman gains, calculated for that velocity. To begin implementing gain scheduling and linear interpolation, a preliminary enhancement is required. This involves modifying the reference table so the bike can have varying velocity references at each (X_{ref}, Y_{ref}) point. The correct velocity reference must be extracted at each time step using the closest point algorithm to achieve the desired speed at each moment of the run. The next step is to implement gain scheduling and validate its effectiveness in simulation to determine if it should be applied to the bikes. Once gain scheduling is validated, the possibility of applying linear interpolation to system matrices and gains will be explored. If feasible, it is implemented in simulation to ensure smoother transitions in gains during velocity changes. After the work in MATLAB/Simulink is completed and validated, it is also replicated in LabVIEW/C.

A. Reference Generation

As introduced in Figure 1, the input to the system is a trajectory matrix that includes $[X_{ref}, Y_{ref}, \psi_{ref}, v_{ref}]$. A function, `Refgeneration.m`, has been created to generate this trajectory matrix based on user input. The user provides a matrix to the function that includes one of the following combinations:

- $[X, Y, t]$: The function calculates the velocity reference based on the given time, $dt = t^+ - t$, between the coordinates (X, Y) and (X^+, Y^+) using the following formula:
$$v = \frac{\sqrt{dx^2 + dy^2}}{dt} \tag{2}$$
where $dx = X^+ - X$ and $dy = Y^+ - Y$.
The heading angle reference is then calculated using:
$$\psi = \arctan\left(\frac{dy}{dx}\right) \tag{3}$$
- $[v, \psi, t]$: In this case, the reference coordinates are assumed to start at the origin, $X_{ref}(0) = Y_{ref}(0) = 0$, and they propagate according to the following equations:
$$X^+ = X - \frac{v}{dt} \cdot \cos(\psi) \tag{4}$$
$$Y^+ = Y - \frac{v}{dt} \cdot \sin(\psi)$$
- $[X, Y, v]$: The only missing reference in this case is the heading, ψ, which is calculated using Equation 3.

The function `Refgeneration.m` can be found in the appendix A.

B. Velocity reference extraction

In Simulink, the velocity reference vector has to pass through the closest point algorithm to extract the correct velocity reference at each time instant based on the closest point index. This ensures that the correct speed is always inputted to the speed controller as reference velocity. The same approach is applied in LabVIEW to achieve the same goal. In LabVIEW, the process begins by taking the trajectory matrix, which includes all the velocity references in one of its columns, transposing the matrix, and then reshaping it into a 1x4N vector containing all the elements of the trajectory matrix. The velocities are then extracted, and the velocity vector is connected to the closest point index to obtain the desired velocity reference consistently. How the velocity reference is selected is illustrated in figure 4.

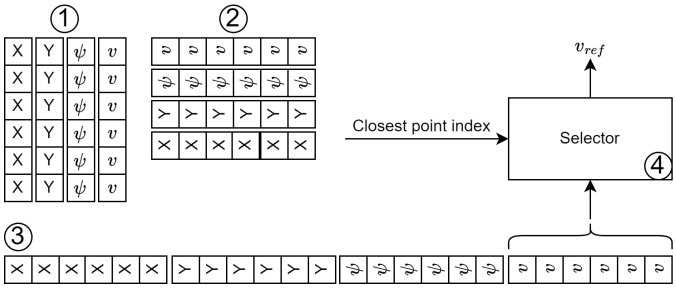


Fig. 4. Explaining how the correct velocity reference is extracted in LabVIEW from the trajectory matrix

C. System adaptation and Gain scheduling

To implement system adaptation and gain scheduling based on velocity, it is essential to evaluate the system matrices and gains that might be influenced by changes in velocity. This process involves examining various subsystems where velocity plays a role to determine if they are affected by velocity variations. The subsystems under consideration include the LQR controller within the trajectory controller, the Kalman estimator, and the transfer function of the $\dot{\psi}_{ref}$ contribution in trajectory control. Each subsystem is analyzed by varying the velocity to observe whether the matrices and gains change accordingly. The analysis reveals that, unlike the other subsystems, the LQR gain remains unaffected by velocity changes. Consequently, further investigation is conducted to understand this behavior.

1) LQR controller: By solving the Riccati equation for the gain K, as illustrated in equations 5 and 6, we will show that LQR gain does not depend on velocity. The state and input matrices A and B are defined as follows:

$$A = \begin{bmatrix} 0 & v \\ 0 & 0 \end{bmatrix}, B = \begin{bmatrix} \frac{lr \cdot v}{lr+lf} \\ \frac{v}{lr+lf} \end{bmatrix}$$

The initial step involves solving the following equation for S:

$$0 = A^T S + SA + Q - SBR^{-1}B^T S \tag{5}$$

The solution to equation 5 yields a 2x2 matrix as shown below:

$$S = \begin{bmatrix} \frac{N_1}{v} & \frac{N_2}{v} \\ \frac{N_2}{v} & \frac{N_3}{v} \end{bmatrix}$$

where N_1, N_2 and N_3 are independent from v.
Once S is determined, the LQR gain K can be calculated using equation 6:

$$K = R^{-1}B^T S \tag{6}$$

which simplifies to:

$$K = \begin{bmatrix} K_1 \\ K_2 \end{bmatrix} = R^{-1} \begin{bmatrix} \frac{lr \cdot v}{lr+lf} & \frac{v}{lr+lf} \end{bmatrix} \begin{bmatrix} \frac{N_1}{v} & \frac{N_2}{v} \\ \frac{N_2}{v} & \frac{N_3}{v} \end{bmatrix}$$

$$= R^{-1} \begin{bmatrix} \frac{lr \cdot N_1 + N_2}{lr+lf} & \frac{lr \cdot N_2 + N_3}{lr+lf} \end{bmatrix}$$

As it is shown above, velocity does not influence the LQR gain K. Consequently, the LQR gain is not considered for gain scheduling.

2) Kalman Estimator and $\dot{\psi}_{ref}$ contribution : By running the system simulation across different velocities, it becomes evident which subsystems are influenced by velocity variations. This is apparent as certain system matrices and gains change in response to velocity adjustments. One such example is the Kalman gain, as shown in Appendix C, where two sets of gains are calculated under identical conditions, differing only in velocity. This verification process was applied to all gains and matrices in the simulation, revealing that only the Kalman estimator and the $\dot{\psi}_{ref}$ contribution in the trajectory control are affected by changes in velocity. Therefore, it is essential to compute the gains and matrices of these subsystems at various velocity reference points using an adjustable step size. This method enables scheduling of the gains and matrices in response to velocity changes. The process is implemented in MATLAB, and the code below shows how different sets of matrices and gains are selected based on the estimated current velocity of the bike.

```
differences = abs(V - V_est);
[~, index1] = min(differences);
K_GPS=K_GPS(index1,:,:);
```

As seen in the code above, the velocity vector V, constructed between the minimum and maximum expected velocities of the run from the trajectory matrix, is subtracted from the estimated velocity. The index corresponding to the smallest difference between the estimated and reference velocities is then identified. This index is used to select the corresponding set of gains and matrices, such as K_{GPS}, the Kalman gain when GPS data is available. This approach ensures that the gains and matrices associated with the closest velocity in the velocity vector are used, scheduling them according to velocity changes.

D. Interpolation

In this context, the gains and system matrices are calculated for different velocities. Rather than selecting the closest pre-calculated gains or matrices and switching abruptly between them as the velocity changes, linear interpolation is employed to smoothly compute the appropriate gains or matrices for the current velocity.

This is achieved by considering both the lower and higher pre-calculated gains/matrices and weighting their influence based on how close the current velocity is to these pre-calculated velocities.

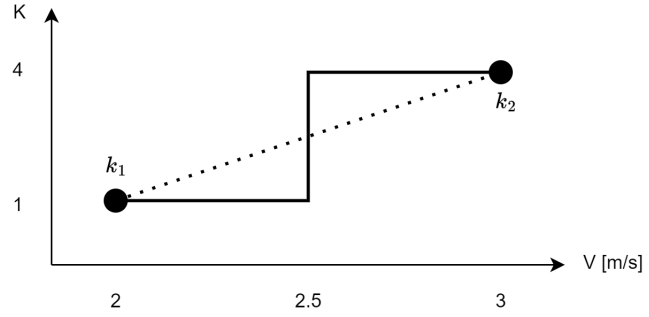


Fig. 5. The figure illustrates an example of gain calculation for two velocities: $v = 2$m/s, represented by k_1, and $v = 3$m/s, represented by k_2. When the current velocity is less than 2.5m/s, the gain is set to k_1. As soon as the velocity exceeds 2.5m/s, the gain abruptly shifts to k_2. This step change can lead to aggressive fluctuations in the gain if the velocity hovers around 2.5m/s. However, by using linear interpolation, the corresponding gain value will lie on the dashed line connecting k_1 and k_2. This approach ensures a smooth transition between gains, reducing abrupt changes.

There are multiple options for implementing the gain/matrices scheduling on the actual bike:

- Online calculation on the hardware: Doing the calculation such as solving the riqatti equation on the hardware using LabVIEW is computationally heavy and requires high experience and knowledge in LabVIEW which is one of the limitations in this project.
- Offline calculation in Matlab: pre-calculation of these gains and matrices in Matlab and then upload them to the hardware and schedule them in LabVIEW. This solution will occupy a part of the hardware memory depending on the step size used in the velocity vector that has been pre-calculated.

Since linear interpolation is used, a larger step size in the velocity vector can be employed, thereby reducing the number of gains/matrices that need to be stored on the hardware. If it can be confirmed that the matrices change linearly with different velocities, it is even be possible to use just two sets of gains/matrices: one corresponding to the lowest expected velocity and one to the maximum expected velocity. The appropriate values can then be interpolated based on the current estimated velocity.

1) Linearity check: The gains and the matrices that need to be scheduled have an added dimension to indicate the velocity used for calculating them.

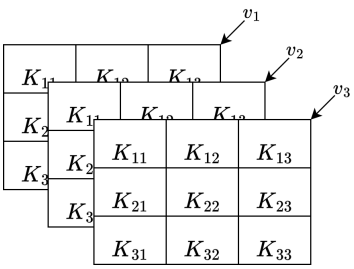


Fig. 6. An illustration of how the gains and matrices have been saved. a unique matrix has been calculated for each velocity in the vector.

A measure of the linearity, MOL, on how K_{jk} changes across all v_i has been developed through discussion and brainstorming. Using the built-in Matlab function `polyfit`, each K_{jk} is fitted to a line, and the sum of the residuals between the data points and the corresponding values on the fitted line is calculated. To make this measure useful, the sum of the residuals is compared to the sum of the K_{jk} values by dividing the two. The result is then multiplied by 100, providing a percentage that indicates how much the fitted line can describe the actual data points.

$$MOL = \left(1 - \frac{\sum |r_i|}{|\sum K_{jk_i}|}\right) \cdot 100 \qquad \sum |r_i| < |\sum K_{jk_i}| \quad (7)$$

The issue with this function arises when the fit line is extremely poor, such that the sum of the residuals exceeds the sum of the data points themselves. In this case, the measure of linearity (MOL) will return a negative value, indicating that the fit line is not a reliable representation of the data.

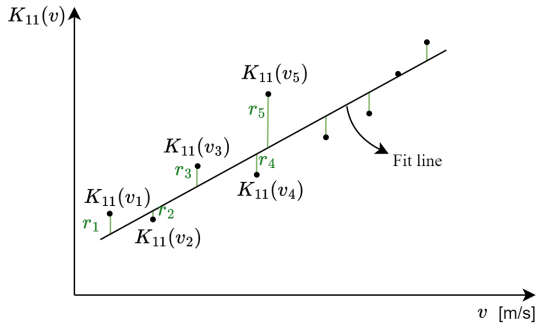
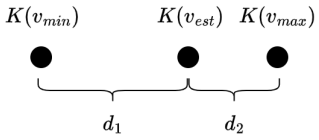


Fig. 7. The figure shows an example of how the gain matrix first element, K_{11}, is changing across all the velocities. The fit line is calculated in Matlab.

2) Implementation : After confirming that the gains and matrices change linearly, it is possible to implement linear interpolation. This allows the use of only the matrices and gains corresponding to the minimum and maximum velocities while still achieving the correct gains/matrices based on the estimated velocity. The linear interpolation is performed according to equation 8 below.

$$K(v_{est}) = \frac{K(v_{max})d_1 + K(v_{min})d_2}{d_1 + d_2} \quad (8)$$



Where $d_1 = v_{est} - v_{min}$, $d_2 = v_{max} - v_{est}$.

Linear interpolation for all scheduled matrices and gains is implemented in both MATLAB/Simulink and LabVIEW and a saturation is added as following

$$K(v_{est}) = K(v_{max}) \qquad v_{est} \geq v_{max}$$
$$K(v_{est}) = K(v_{min}) \qquad v_{est} \leq v_{min}$$

In LabVIEW, a subVI is created specifically to perform interpolation on matrices and gains, as detailed in Appendix B.

III. Results

A. Validation and the result of MOL

Since the measure of linearity (MOL) was developed during discussions, it requires a clear motivation. Rather than deriving a theoretical justification for why the MOL equation is appropriate for assessing how well the fit line represents the data points, a test has been conducted using third-, second-, and first-degree polynomials to observe the resulting MOL values. This test serves as an indicator of how effectively the MOL equation can evaluate the linearity of the gains/matrices.

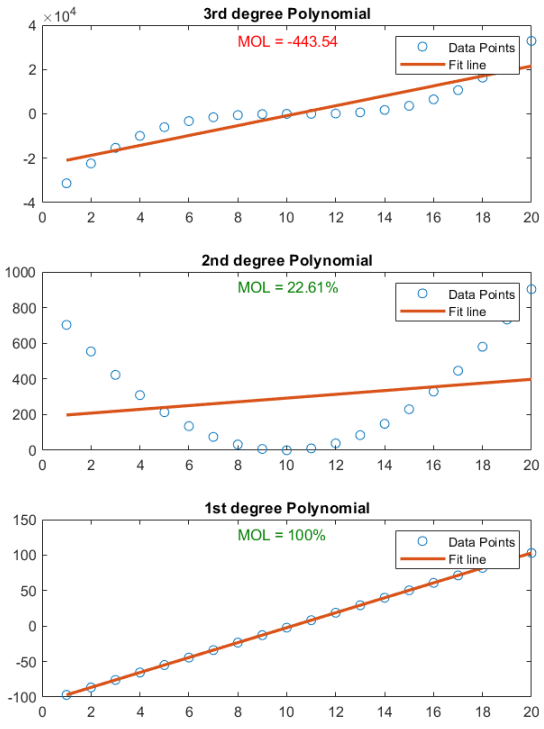


Fig. 8. The figure shows the test of using the MOL function with different polynomials.

Figure 8 is an indicator that the MOL (Measure of Linearity) function is an effective tool for assessing the linearity of matrix and gain changes as the velocity varies. Using this method, all scheduled matrices and gains were evaluated for linearity with respect to velocity's linear growth. The results showed that all had MOL values very close to 100%, except for one element within the Kalman gain when GPS data are available, specifically K_{GPS}, shown in figure 9.

This element was therefore further analyzed and plotted. As shown in Figure 10, the values do not exhibit linear behavior; however, they remain very close to zero. Consequently, they can be considered negligible and, therefore, not seen as non-linear changes.

```
MOL =

  100.0000       NaN       NaN       NaN       NaN       NaN  100.0000
       NaN   99.9820   99.9975       NaN       NaN       NaN       NaN
       NaN       NaN    9.0909   99.9934   83.2897   95.8697       NaN
       NaN       NaN       NaN  100.0000  100.0000       NaN       NaN
       NaN   96.9623   96.9621       NaN   90.8458   99.8619       NaN
       NaN   99.8621   99.8623   95.8533   96.7970   99.9493       NaN
  100.0000       NaN       NaN       NaN       NaN       NaN  100.0000
```

Fig. 9. The element which according to MOL, has an MOL of 9%. It means that this element is not changing linearly with velocity growth. Note: The NaN elements correspond to elements which are always 0 no matter the velocity.

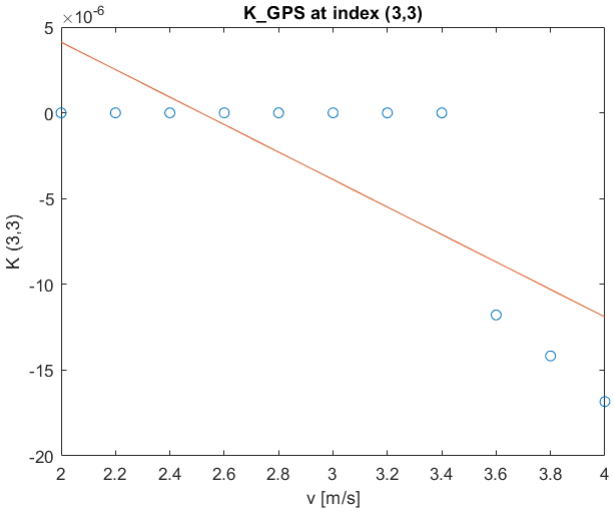


Fig. 10. The figure illustrates how the values for $K_{GPS}(3,3)$, across different velocities are all close to zero, indicating that it should not be considered a non-linearly changing element. The values for $K_{GPS}(3,3)$ shown as blue circles while the fit line is shown in red.

B. The result of running the bike on simulation

The results of this work were primarily validated through simulations in Matlab/Simulink, as issues in previous work caused errors in extracting the velocity reference using the closest point index. The index is always stuck at value 1 due to unknown reasons. The references extraction process is shown in Figure 4. Despite this, the gain scheduling and linear interpolation were successfully implemented both in simulation and the actual bike in LabVIEW and validated in the lab using a bike roller. The reference velocity was changed manually, and the system's matrices were adapted correctly, confirming that the newly implemented features are functioning as expected.

For further validation, simulations involved running the bike on a custom trajectory. First, the bike traveled in a straight line for 50 meters at a constant speed of 3 m/s. Then, it followed a circular path with a diameter of 40 meters, while varying its speed sinusoidally between 2 and 4 m/s.

The simulation results are as follows:

1) Case 1 - Gains and matrices based on constant velocity: Before implementing gain scheduling and system adaptation, the matrices and gains were calculated based on a constant velocity of 3 m/s. At this speed, the bike was able to balance

and follow the trajectory. However, when the velocity deviated from 3 m/s, both the lateral and heading errors increased. The farther the velocity strayed from 3 m/s, the larger the errors became. Interestingly, at lower velocities around 2 m/s, the errors were even more pronounced compared to those at 4 m/s, despite both being equally distant from 3 m/s. This is due to the bike being less stable at lower speeds.

As shown in Figure 11, the lateral error oscillates between -98 cm at a velocity of 2 m/s and up to 80 cm at 4 m/s. Similarly, the heading error oscillates between -6.7 degrees at 2 m/s and 1.9 degrees at 4 m/s.

The reference trajectory and the estimated position during the run are provided in Appendix D.

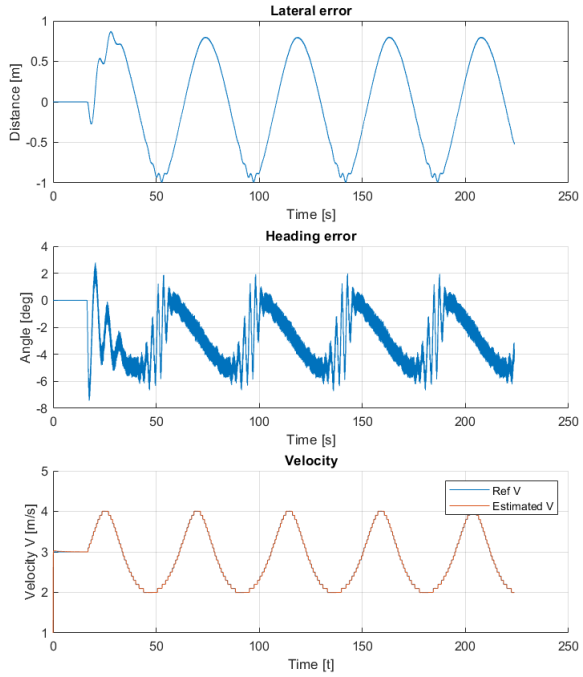


Fig. 11. The lateral error e_1 and the heading error e_2 during test case 1 before implementing scheduling. The bike runs straight at a constant speed and then follows a circular path where the velocity changes sinusoidally between 2 and 4 m/s.

2) Case 2 - Gains and Matrices Scheduled: In this case, the matrices and gains are scheduled. Five sets of gains and matrices were calculated for velocities of 2, 2.5, 3, 3.5, and 4 m/s, which were applied based on the current velocity, as explained in Section II-C2. As a result, the bike remains stable and follows the trajectory with significantly reduced errors compared to Case 1. This improvement is attributed to the system's enhanced ability to capture the bike's dynamics by adapting the heading rate contribution transfer function and improving the accuracy of state estimation.

As illustrated in Figure 12, the lateral error now oscillates between 37 cm at a velocity of 2 m/s and 5 cm at 4 m/s, which is significantly lower than in Case 1. Meanwhile, the heading error oscillates between -0.4 and -5.5 degrees, representing a slight improvement compared to Case 1.

The reference trajectory and the estimated position during the run are provided in Appendix D.

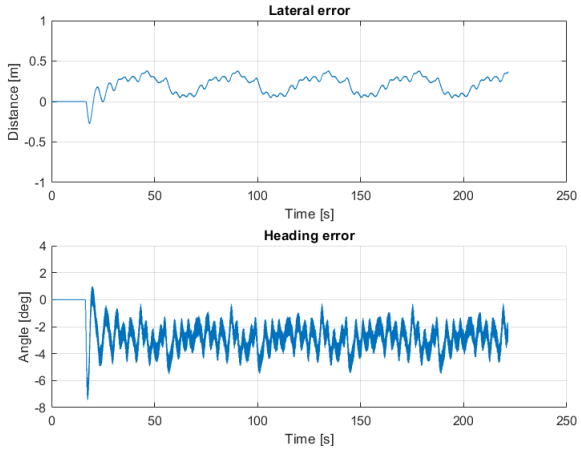


Fig. 12. The lateral error, e_1, and the heading error, e_2, during test case 2 show a significant improvement in the lateral error compared to case 1. However, both errors exhibit considerable oscillations caused by sudden changes in the gains and matrices.

3) Case 3 - Gains and Matrices Scheduled and adding Linear Interpolation: In case 2 the lateral error was changing jaggedly due to the abrupt changes in the Matrices and gains while changing the velocity. Introducing the linear interpolation smoothens the lateral error curve and even reduces the heading error.

The lateral error now oscillates between 6 and 29 cm. Meanwhile, the heading error oscillates between -4.2 and -1.7 degrees very similar to the test case 3.

The reference trajectory and the estimated position during the run are provided in Appendix D.

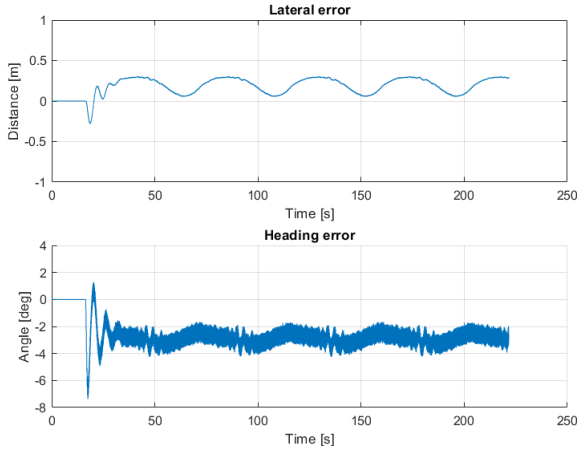


Fig. 13. The lateral error e_1 and heading error e_2 during test case 3 show much smoother behavior after the addition of linear interpolation, although the overall reduction in errors is minimal compared to case 2.

4) Case 4 - Gains and Matrices Scheduling with Linear Interpolation Using Only Min/Max Matrices: As discussed in section II-D2, the matrices and gains exhibit linear behavior across all velocities. By applying linear interpolation based solely on the matrices and gains corresponding to the minimum and maximum velocities, the system achieves performance similar to Case 3.

The lateral error now oscillates between 0 and 24 cm, while the heading error ranges from -4.2 to -1.6 degrees, showing similar results as in test case 3.

The reference trajectory and the estimated position during the run are provided in Appendix D.

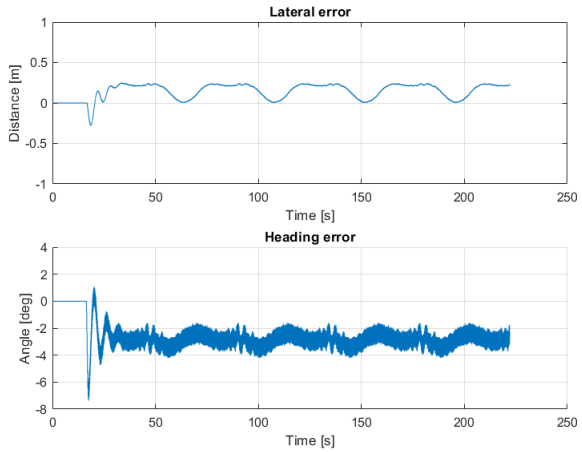


Fig. 14. The lateral error e_1 and heading error e_2 in test case 4 remain consistent with those observed in case 3. This confirms that using only the maximum and minimum matrices and gains is sufficient, eliminating the need for the multiple sets of matrices employed in case 3.

IV. Future work:

1) The reference index problem on the actual bike: The position reference is determined in a C code using the closest point algorithm, and the references $(X_{ref}, Y_{ref}, Psi_{ref}, idx)$ are then sent to LabVIEW. While the references $(X_{ref}, Y_{ref}, Psi_{ref})$ are selected correctly, allowing the bike to follow the correct trajectory, the output for the index (idx) is stuck at $idx = 1$. When this issue is resolved, the bike will be able to pick the velocity reference from the trajectory matrix properly.

2) Implementing ILC: Due to the project's priorities and time constraints, we were unfortunately unable to focus on the next step, which would involve implementing Iterative Learning Control (ILC). Implementing ILC would enable the bike to run multiple iterations, gradually reducing errors by learning from previous iterations and adjusting the reference signals accordingly until convergence is achieved. The first step in this process is to add Δ_{ref} to the reference velocity and reference steering angle, which are the two interesting signals in implementing ILC. This step has already been completed in our project, but it requires multiple simulations to determine the extent of Δ_{ref}'s influence on the signals and to establish the threshold at which it begins to negatively impact the reference signals. Following this, the ILC algorithms must be studied and implemented to achieve smoother and more accurate path tracking. We recommend this as a key focus for future work on the Autobike project.

V. Conclusion:

This project successfully implemented gain scheduling/system adaptation and linear interpolation based on the velocity of a self-driving bike to enhance its adaptive trajectory control. The simulation results demonstrated the effectiveness of these techniques in improving the bike's stability and path-tracking accuracy across varying speeds. By adapting system matrices and gains according to real-time velocity changes, the bike achieved smoother transitions and reduced errors in lateral and heading deviations. Additionally, the incorporation of linear interpolation further refined the system's response, minimizing abrupt changes and ensuring more consistent performance.

These improvements were validated through several test cases, confirming that the new control strategies significantly outperform those based on constant velocity assumptions. The reduction in errors, particularly in scenarios with fluctuating speeds, highlights the robustness of the adaptive approach.

Overall, this study makes a substantial contribution to the development of autonomous vehicles by enhancing the adaptive control mechanisms needed for reliable and accurate trajectory following in self-driving bikes.

References

[1] G. Wen and J. Sjöberg, "Lateral control of a self-driving bike," 2022, https://ieeexplore.ieee.org/stamp/stamp.jsp?arnumber=9986548.

APPENDIX

A. *Refgeneration.m*

```
% This fucntion is used to generate the needed references in order to
% control the bike. The user need to attach the trajectory that the bike
% will follow as a csv file, inclouding three columns.
% these theree columes have to be enough to output
% [X_ref,Y_ref,V_ref,Psi_ref,t] at each sample.
% Any combination in form of [x,y,t] [v,t,psi] and [x,y,v] will work.
% any other combination data will lead to an error.
% How to use:
% - The user needs to provide the correct order of the data combination (in the given trajectory CSV file)
    as the first parameter.
% - The user needs to provide the path directory of the CSV file as the
% second parameter.
% The function output will be [X_ref,Y_ref,V_ref,Psi_ref,t].
% Examples:
% [x,y,psi,v,t]=Refgeneration({'x','y','t'},'Traj_ref_test\traj1.csv');
% [x,y,psi,v,t]=Refgeneration({'y','t','x'},'Traj_ref_test\traj2.csv');
% [x,y,psi,v,t]=Refgeneration({'v','t','psi'},'Traj_ref_test\traj3.csv');
% [x,y,psi,v,t]=Refgeneration({'x','y','v'},'Traj_ref_test\traj4.csv');
% Authors: Armin Khorsandi & Ayman Alzein summer 2024

function [x,y,psi,v,t] = Refgeneration(layout,filePath)

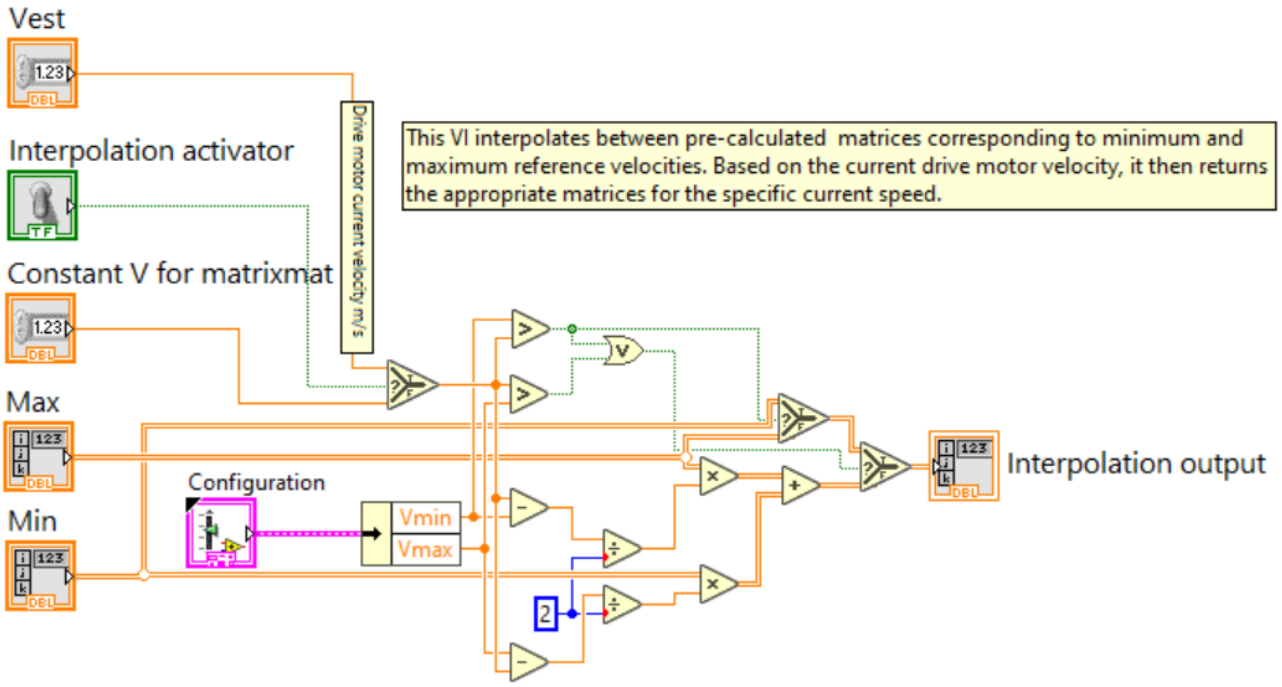
    if isequal(layout, {'x', 'y', 't'})|| isequal(layout, {'t', 'y', 'x'})|| isequal(layout, {'x', 't', 'y'
        })|| isequal(layout, {'t', 'x', 'y'})|| isequal(layout, {'y', 't', 'x'})|| isequal(layout, {'y', 'x
        ', 't'})
        i = 1;
    elseif isequal(layout, {'v', 'psi', 't'})|| isequal(layout, {'v', 't', 'psi'})|| isequal(layout, {'t',
        'psi', 'v'})|| isequal(layout, {'t', 'v', 'psi'})|| isequal(layout, {'psi', 'v', 't'})|| isequal(
        layout, {'psi', 't', 'v'})
        i = 2;
    elseif isequal(layout, {'x', 'y', 'v'})|| isequal(layout, {'v', 'y', 'x'})|| isequal(layout, {'x', 'v',
         'y'})|| isequal(layout, {'v', 'x', 'y'})|| isequal(layout, {'y', 'v', 'x'})|| isequal(layout, {'y'
        , 'x', 'v'})
        i = 3;
    else
        error('Not a valid input layout. Please attach one csv file path with only three columns. For more
            info, please refer to the function description.')
    end

    table_ref = readtable(filePath);
    table_ref = renamevars(table_ref, 'Var1', layout(1));
    table_ref = renamevars(table_ref, 'Var2', layout(2));
    table_ref = renamevars(table_ref, 'Var3', layout(3));

    if i==1
        t=table_ref.t;
        x=table_ref.x;
        y=table_ref.y;
        psi=zeros(length(x),1);
        v=zeros(length(x),1);
        for i=1:length(x)-1
            dx=x(i+1)-x(i);
            dy=y(i+1)-y(i);
            dt=t(i+1)-t(i);
            psi(i)=atan2(dy,dx);
            v(i)=sqrt(dx^2+dy^2)/dt;
        end
        psi(end)=psi(end-1);
        v(end)=v(end-1);
    end
    if i==2
        t=table_ref.t;
        psi=table_ref.psi;
        v=table_ref.v;
        x=zeros(length(v),1);
        y=zeros(length(v),1);
        for i=1:length(v)-1
            dt=t(i+1)-t(i);
            x(i+1)= x(i)+(v(i)/dt)*cos(psi(i));
            y(i+1)= y(i)+(v(i)/dt)*sin(psi(i));
```

```
        end
    end
     if i==3
        v=table_ref.v;
        x=table_ref.x;
        y=table_ref.y;
        t=zeros(length(x),1);
        psi=zeros(length(x),1);
        for i=1:length(x)-1
            dx=x(i+1)-x(i);
            dy=y(i+1)-y(i);
            psi(i)=atan2(dy,dx);
            t(i+1)=sqrt(dx^2+dy^2)/v(i);
        end
        psi(end)=psi(end-1);
     end
end
```

B. Interpolation sub VI in Labview



C. Example of a gain matrix that changes by changing the velocity.

$$\underbrace{\begin{bmatrix} 0.2227 & 0 & 0 & 0 & 0 & 0 & 0.0096 \\ 0 & 0.2419 & 0 & 0 & 0 & 0.0085 & 0 \\ 0 & 0.2205 & 0 & 0 & 0.0001 & 0.0171 & 0 \\ 0 & 0 & -0.0001 & 0.0100 & 0 & 0 & 0 \\ 0 & 0 & 0 & 1.0000 & 0.0002 & 0.0589 & 0 \\ 0 & 0 & -0.0001 & 0 & 0.0031 & 0.9947 & 0 \\ 0.0001 & 0 & 0 & 0 & 0 & 0 & 0.8541 \end{bmatrix}}_{\text{Kalman gain at } v{=}2\ m/s} \quad \underbrace{\begin{bmatrix} 0.2227 & 0 & 0 & 0 & 0 & 0 & 0.0096 \\ 0 & 0.2599 & 0 & 0 & 0.0001 & 0.0167 & 0 \\ 0 & 0.2186 & 0 & 0 & 0.0002 & 0.0336 & 0 \\ 0 & 0 & -0.0001 & 0.0100 & 0 & -0.0002 & 0 \\ 0 & 0 & -0.0001 & 1.0000 & 0.0016 & 0.2553 & 0 \\ 0 & 0 & -0.0005 & 0 & 0.0060 & 0.9778 & 0 \\ 0.0001 & 0 & 0 & 0 & 0 & 0 & 0.8541 \end{bmatrix}}_{\text{Kalman gain at } v{=}4\ m/s}$$

D. The reference trajectory and the estimated position during the run for all four test cases. A noticeable improvement is observed when comparing Case 1 to Case 2. However, the improvements in Cases 3 and 4 become harder to visualize in the figures below, as they display the entire trajectory in meters. Consequently, improvements of just a few centimeters are difficult to discern in these plots.

